2019 EXAMINATIONS

Part II

COMPUTING AND COMMUNICATIONS [2.5 Hours]

SCC.312 Languages and Compilation

Candidates are asked to answer THREE questions from FOUR; each question is worth a total of 25 marks.

Use a separate answer book for each question

Question 1

1.a Let M be a deterministic finite state machine (DFSM) whose alphabet is $\{0, 1\}$. Which of the following statements about M are valid?

i. M accepts 0.

ii. If M has a final state, then the language recognized by M is not empty, that is, $L(M) \neq \emptyset$.

iii. If M accepts 0 and 00, then it also accepts 000.

iv. If $L(M) = 0^+$, then M has more than one state.

v. If $\varepsilon \in L(M)$, then M's initial state is also a final state.

[5 marks]

1.b Draw a smallest (in terms of the number of states) DFSM K such that the language recognized by K, that is, $L(K) = \{w \mid w \text{ ends with a } 0\}$. Assume that the alphabet is $\{0, 1\}$.

[4 marks]

1.c Consider the nondeterministic finite state machine (NFSM) ***N*** in Figure 1 (assume its alphabet is $\{0, 1\}$).

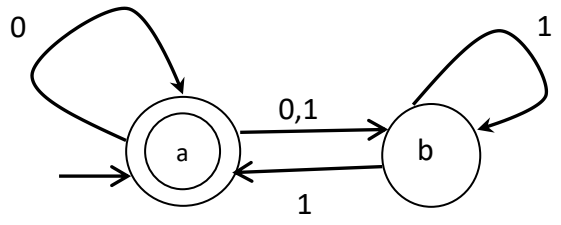

Figure 1: NFSM ***N***

For each of the following strings, give the set of states ***N*** is in after reading the string and also indicate whether the string is accepted or rejected.

i. 0101

ii. 1011

[8 marks]

Question 1 continues on next page…

Question 1 continued.

1.d Consider the NFSM ***N*** in Figure 1 (again, assume its alphabet is {0, 1}). Suppose we converted it into a deterministic version *D* by applying the subset construction algorithm that we studied in class. The algorithm tells us that the set of states of D is $\{\emptyset, \{a\}, \{b\}, \{a, b\}\}$. By applying the algorithm further, answer the following questions about *D*.

i. Give the initial state of D.

ii. Give the set of final states of D.

iii. Give the state *D* transitions into upon seeing 1 in $\{a, b\}$.

iv. Give the state *D* transitions into upon seeing 0 in $\{a\}$.

[8 marks]

Total 25 marks

Question 2

2.a Which of the following statements are true?

i. Every string in the language of regular expressions defines a language that may or may not be regular.

ii. Every subset of a regular language is regular

iii. To prove a language is regular, it is sufficient to give an FSM for it

iv. Context-free languages are a subset of regular languages

v. $\Sigma^+ \subseteq \Sigma^*$

[5 marks]

2.b Give regular expressions for the following languages. Assume the alphabet is $\{0, 1\}$.

i. $\{w \mid w \text{ contains a single } 0\}$

ii. $\{w \mid w \text{ begins and ends with } 1\}$

[6 marks]

2.c Let G_1 be the following grammar.

$$P \rightarrow \varepsilon$$
$$P \rightarrow 0$$
$$P \rightarrow 1$$
$$P \rightarrow 0P0$$
$$P \rightarrow 1P1$$

Let $L(G_1)$ be the language of G_1. Which of the following statements are true?

i. $\{1, 10, 101, 1010\} \subseteq L(G_1)$

ii. $\{\varepsilon, 0, 1, 10\} \subseteq L(G_1)$

iii. $\{\varepsilon, 0, 1, 101\} \subseteq L(G_1)$

iv. $\{0, 1, 101, 010\} \subseteq L(G_1)$

[6 marks]

(Getting all the statements correct gets you full marks; zero otherwise)

Question 2 continues on next page...

Question 2 continued.

2.d Characterize succinctly in English the kinds of strings G_1 generates.

[2 marks]

2.e Let grammar $G_2 = (V, \Sigma, R, S)$ where $V = \{T\}$ is the set of variables; $\Sigma = \{0, 1\}$ is the set of terminals; and S, the start variable, is T. Give the set of substitution rules R such that the language of the grammar is $\{0^n 1^n \mid n \geq 0\}$.

[6 marks]

Total 25 marks

Question 3

3.a Define and provide an example of both explicit and implicit type conversion.

[3 marks]

3.b Explain how an Intermediate Language (IL) stage potentially limits the functionality of a compiler?

[4 marks]

3.c

i. Using the following grammar and input string, show each step of a bottom-up (LR) parse. At each step show the state of the stack.

[11 marks]

Grammar:

```
S --> ID = EXPR

EXPR --> ID | ID OP EXPR | ( EXPR )

OP --> + | - | *

ID --> a | b | c | d
```

Input:

“a = b * (c + d)”

ii. Explain what is meant by ‘back tracking’ with regard to compilers.

[2 marks]

iii. Which rule in the grammar used for 3.c.i would potentially cause significant back-tracking in our bottom-up parser? Explain how and be specific.

[2 marks]

3.d.

i. Define the key difference between a macro-assembler and a compiler

[1 mark]

ii. Provide an example of an operation traditionally done by a compiler that could be done by a macro-assembler, and where it could fail

[2 marks]

Total 25 marks

Question 4

4.a Explain the advantages of having an Intermediate Language (IL) stage in a compiler suite.

[3 marks]

4.b Most programming languages are "whitespace agnostic" - what does this mean, and how would a lexical analyser for such a language handle whitespace?

[2 marks]

4.c In a language using structure equivalence to determine type equality, explain how the following pseudocode definition might cause problems.

```
type node is record
    value : integer
    next : pointer to node
end record
```

[3 marks]

4.d Here is a grammar G, with terminals {v, m, t}, non-terminals (Z, Q, R} and Z is the distinguished symbol.

```
Z -> Q v R
Z -> R
Q -> m R
R -> t
```

Build the Action and Goto tables for this grammar using the procedure below, as taught in the lectures:

- Step 1: Augment the grammar. [1 mark]
- Step 2: Build the item sets. [8 marks]
- Step 3: Generate the state transition table. [6 marks]
- Step 4: Produce the Action and Goto Tables. [2 marks]

[17 marks]

Total 25 marks

--- End of Paper ---